



LLM活用入門から応用まで： プロンプト設計・LoRA・RAG

ヒューマンインタフェースシンポジウム 2026
ファムフォーロン、三林亮太

自己紹介



基本情報

- ファムフーロン
- 筑波大学 図書館情報メディア系 特任助教

研究分野

- 情報検索、機械学習

自己紹介



三林 亮太
(Ryota Mibayashi)

所属：神戸大学 国際文化学研究科 助教



産業技術総合研究所 人工知能センター 招聘研究員



専門分野：自然言語処理、情報検索、音楽情報処理

所属学会：日本データベース学会、情報処理学会

京都未踏

京都未踏的人材発掘・育成プログラム

これまでの経歴等：

- 2026年6月 京都未踏的人材発掘・育成事業PM
- 2025年7月 覚醒プロジェクト 採択
- 2023年5月 公益財団法人クマ財団 7期生クリエイター
- 2022年6月 IPA未踏IT人材発掘・育成事業 未踏スーパークリエイター

覚醒
プロジェクト



本チュートリアル概要

- ◆ LLMの基本から、プロンプト設計、RAG、LoRAによるファインチューニングまでをハンズオンで学びます。
- ◆ 各パートで理論と実践をバランスよく学習。
- ◆ 最終的には自分のデータやタスクに合わせたLLM活用力を身につけることを目指します。

キーワード

Prompt

RAG

ReAct

LoRA

BERT

GPT-2

アジェンダ（時間配分）

1.Introduction : 5分

2.プロンプト設計 : 5分

3.RAG : 30分

4.ReAct : 10分

5.LoRA : 40分

6.まとめ : 5分

1

Introduction

(5分)

LLMとは何かを理解し、
本チュートリアル全体像を把握します

LLMとは？

- ◆ Large Language Model（大規模言語モデル）の略
- ◆ 大量のテキストデータを学習し、
人間のように自然な文脈を生成・理解できるモデル
- ◆ 代表例：GPT、Llama、Gemma、Claudeなど



LLMでできること



文章生成・要約・翻訳・質問応答



アイデア出し・ブレインストーミング



プログラミング支援・コード生成

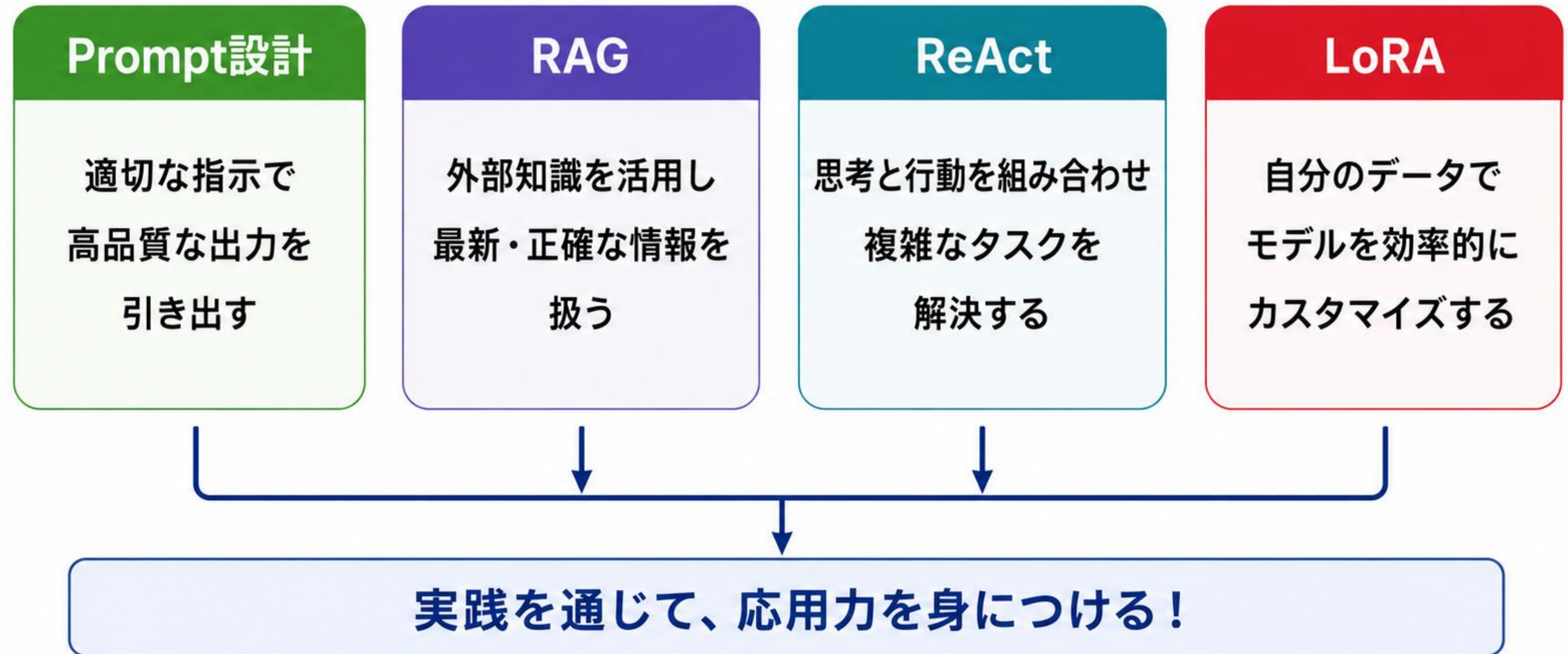


データ分析や説明



業務の自動化・効率化 など

本チュートリアルで学ぶこと



2

Prompt設計

(5分)

効果的なプロンプトの作り方を学びます

プロンプトとは？

- ◆ LLMに対する**指示文**や**質問文**のこと
- ◆ プロンプトの書き方次第で、出力の**質が大きく変わる**
- ◆ **明確**で具体的な指示が**重要**

悪い例

日本について教えて。

良い例

日本の首都はどこですか？人口は？
簡潔に2-3行で教えてください。

プロンプト設計の代表的手法

Zero-Shot: 例を与えずに指示する

One-Shot: 1つの例を与える

Few-Shot: 複数の例を与える

Chain of Thought (CoT): 思考のステップを明示する

ReAct: 思考と行動を組み合わせる

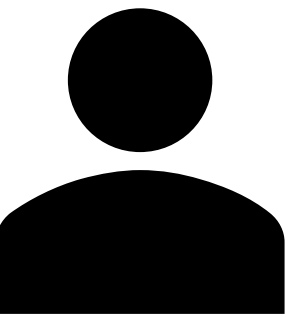
RAG: 外部知識を参照する

Zero-Shot

例を与えずに、直接質問や指示をする方法

Q: 日本首都はどこですか？

A:



Few-shot（複数例を与える）

複数の例を与えることで、より安定した出力に

Q: フランスの首都はどこですか？

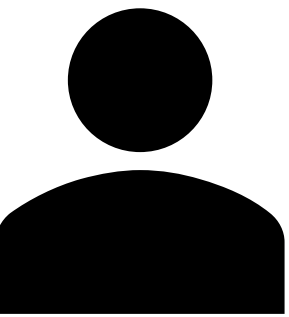
A: パリ

Q: ドイツの首都はどこですか？

A: ベルリン

Q: 日本の首都はどこですか？

A:



CoT (Chain of Thought)

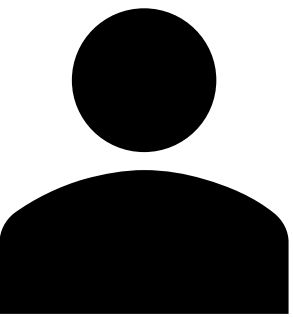
思考のステップを明示することで、複雑な問題の精度が向上

Q: $25 + 17$ は？

A: まず25と17を分けて考えます。

$25 + 10 = 35$ 、 $35 + 7 = 42$ 。

したがって、答えは42です。



③

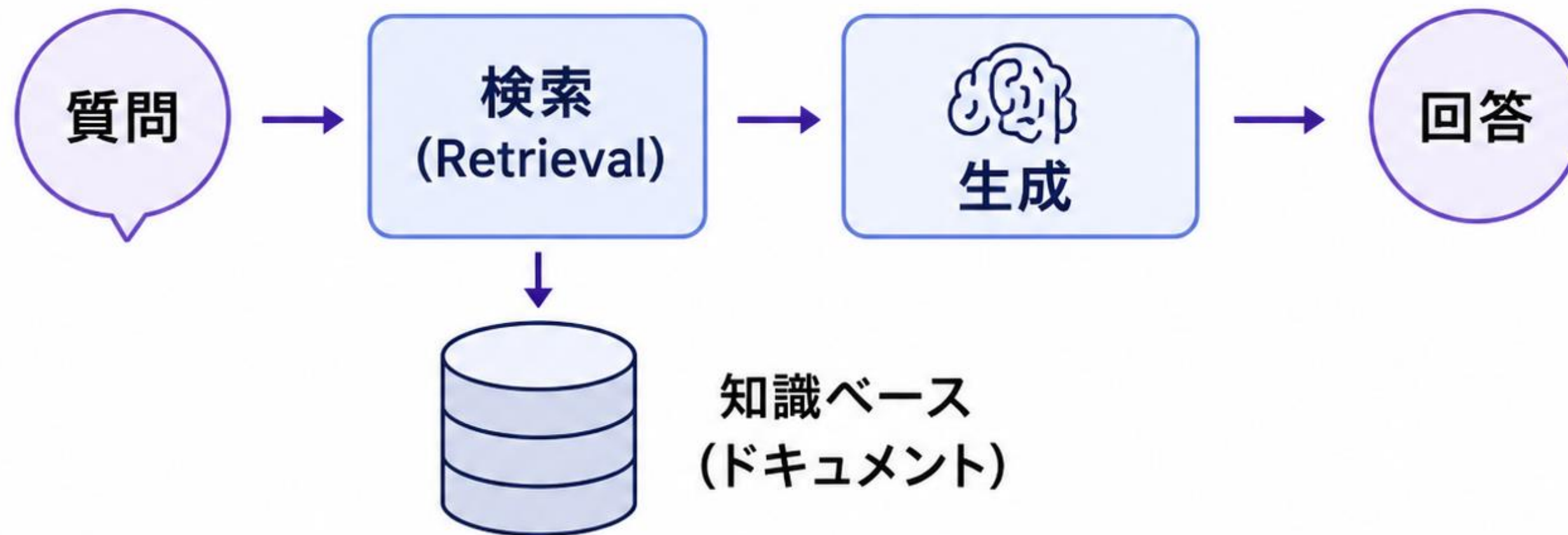
RAG

(20分)

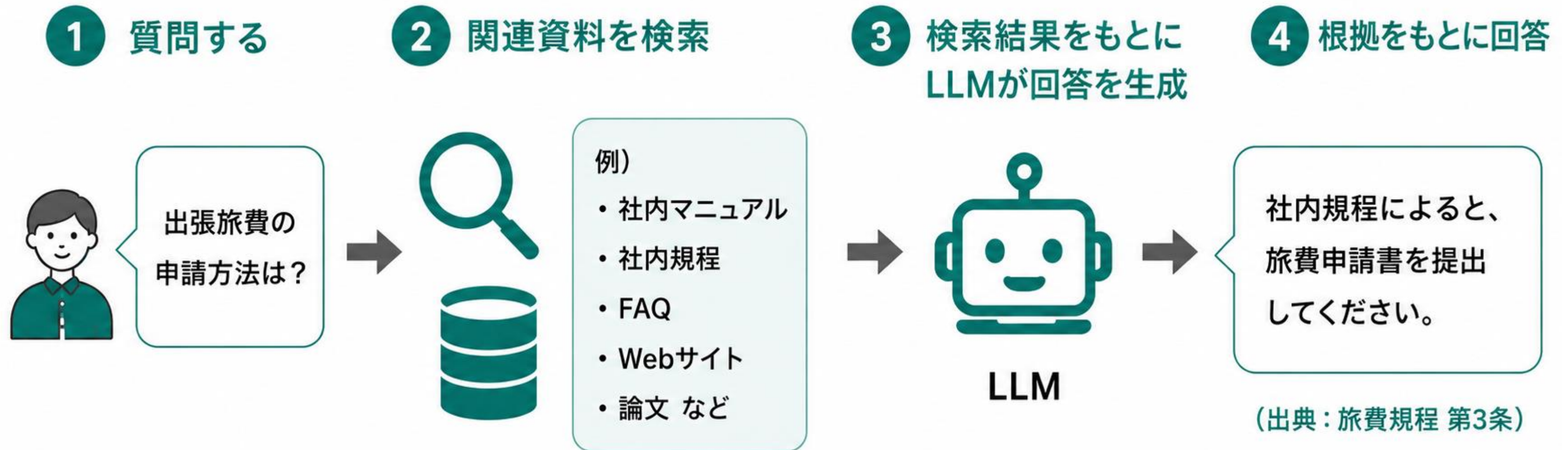
外部知識を活用して、
より正確で最新の回答を生成します

RAGとは？

- ◆ Retrieval Augmented Generation の略
- ◆ 外部の知識ベースから関連情報を検索し、その情報をプロンプトに与えて生成
- ◆ LLMの「知らないこと」や「最新情報」の問題を解決



RAGの流れ



RAGのメリット



最新情報を
使える



社内情報を
活用できる



根拠を
示せる



ハルシネーションを
減らせる

ベクトル検索の仕組み

- ◆ ドキュメントやクエリをベクトルに変換
- ◆ ベクトルの類似度で関連ドキュメントを検索
- ◆ 一般的な手法：コサイン類似度

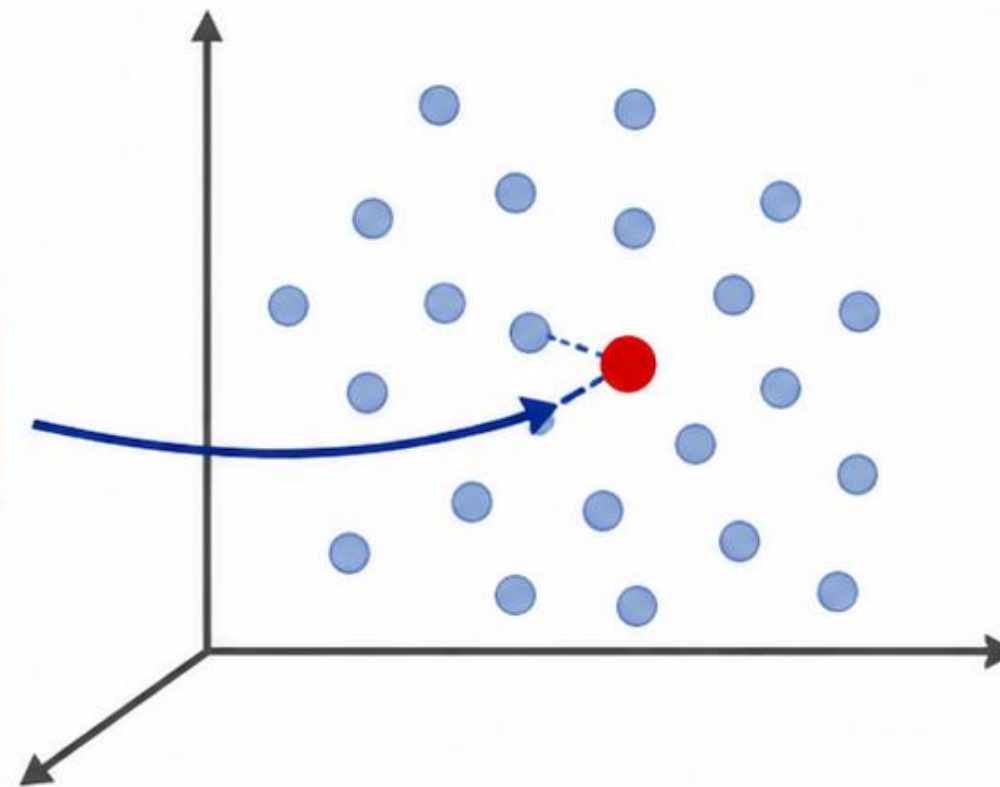
① クエリをベクトル化

クエリ（質問）を
ベクトルに変換

クエリベクトルの例
[0.12, -0.85, ...]

② ベクトル空間で検索

ベクトルの類似度（距離）で
関連するドキュメントを検索



③ 類似度の高いドキュメントを取得

類似度が高い（スコアが高い）
ドキュメントを上位から取得

検索結果（類似度スコアの例）

📄	Doc A	0.92
📄	Doc B	0.85
📄	Doc C	0.78
	⋮	

RAGの効果（例）

✕ RAGなし（LLMのみ）

Q：2024年のノーベル物理学賞の受賞者は？

A：私は2024年の情報を持っていません。



✓ RAGあり（外部情報を参照）

Q：2024年のノーベル物理学賞の受賞者は？

A：2024年のノーベル物理学賞は、ジョン・ホップフィールド氏とジェフリー・ヒントン氏が受賞しました。

RAG Tutorial



3-1

RAG-環境準備

(10分)

ハンズオンのための環境を整えます

3-1-a Google Colabの準備

まずは『動く状態』を作る。中身はまだ理解しなくてOK

1

1. GPUに変更

ランタイム → ランタイムのタイプを
変更 → T4 GPU

2

2. 準備セルを1回実行

!pip install ... / 関数を読み込む

3

3. 表示を確認

『準備用の関数を読み込みました。』が
出ればOK

4

4. 困ったら

時間がかかることがある / エラー文を読む

Google Colab

0. 準備

1. 今日の質問

2. RAGなし

3. 資料を用意

4. 検索

5. RAGあり

6. 比較

ここは『準備』だけ

<https://colab.research.google.com/drive/1XdL-mBGYtATHcvzvMvYze8zDf2mhveXD?usp=sharing>

この段階で大事なのは、まず実行環境を整えること

3-2

RAG-ハンズオン内容

(10分)

実際の操作を通して RAG の活用方法を学びます

3-1-b つまづきポイントと対処

止まっても、どこで止まったかが分かれば理解できる

① インストールが長い

- 最初の1回だけ時間がかかる
- 終わるまで待つ
- 再実行は1回ずつ

② モデル読み込みが遅い

- 初回ロードは数分かかる
- GPU設定を確認
- 途中で切れたら再実行

③ Web取得できない

- 通信が不安定でも大丈夫
- ノートブック内の予備データで続行
- 『通信できなかったため…』と出ても先へ進める



見れば理解できる

資料に沿って見れば、実行できなくてもRAGの流れは理解できる



目印:『資料の先頭』や『準備用の関数を読み込みました。』が出ればOK

3-2-b Step 1 今日の質問を決める

最後まで同じ質問を使う

Notebookでやること

```
question =  
"ヒューマンインタフェース  
シンポジウム2020はどこで  
開催されましたか？"  
  
print("今日の質問:")  
print(question)
```

このセルで分かること

- 今日は1つの質問だけで進める
- 質問をPythonの変数に入れているだけ
- この質問をRAGなし / RAGありで共通に使う

質問を固定することで、後で「条件の違い」を比べやすくなる

3-2-c Step 2 RAGなしで聞いてみる

まずは資料を渡さずにLLMへ質問する

Notebookでやること

```
generator = load_llm()

answer_without_rag =
    ask_without_rag(generator, question)

print("==== RAGなし ====")

print(answer_without_rag)
```

見るポイント

- まだ資料は渡していない
- 回答の正しさより
条件の違いに注目

大事な理解

この時点で、LLMは
この後に読むHTML資料を
見ていない

ここでは『資料なしで答えている』ことが分ければOK

3-2-d Step 3 資料を用意する

Webページを読み込み、あとで使える形にする



RAGのポイント

LLMが最初から知っている
必要はない。必要な資料を、
あとから使わせる

通信トラブルでも続行

- Web取得に失敗したら予備データを使う
- 『資料の先頭:』が表示されればOK

`urlretrieve` → `load_text_from_html` → `document_text`

3-2-e Step 4 質問に近い資料を探す

3-2-e

Retrieval = 資料の中から根拠候補を探す



Notebookでやること

```
chunks = split_text(document_text)
embedding_model = load_embedding_model()
chunk_embeddings = create_embeddings(embedding_model, chunks)
search_results = search_similar_chunks(..., top_k=3)
```

参加者に見てもらうもの

- 資料はいくつに分かれたか
- Top 3に『会場』があるか
- スコアは目安で見る

まだしていないこと

ここではまだ生成しない。検索結果に根拠が見つかったら次にLLMへ渡す。

3-2-f ここで何が起きた？

検索はしたが、まだ生成はしていない

いま行った処理

質問



関係ありそうな
資料を探す

まだしていないこと

まだLLMには答えさせていない。
検索結果に答えの根拠が見つかったら、
次にその資料をLLMへ渡す。

検索結果の見方

資料を ○個のかたまりに分けました。

--- 検索結果 1 ---

... 会場：グランシップ ...

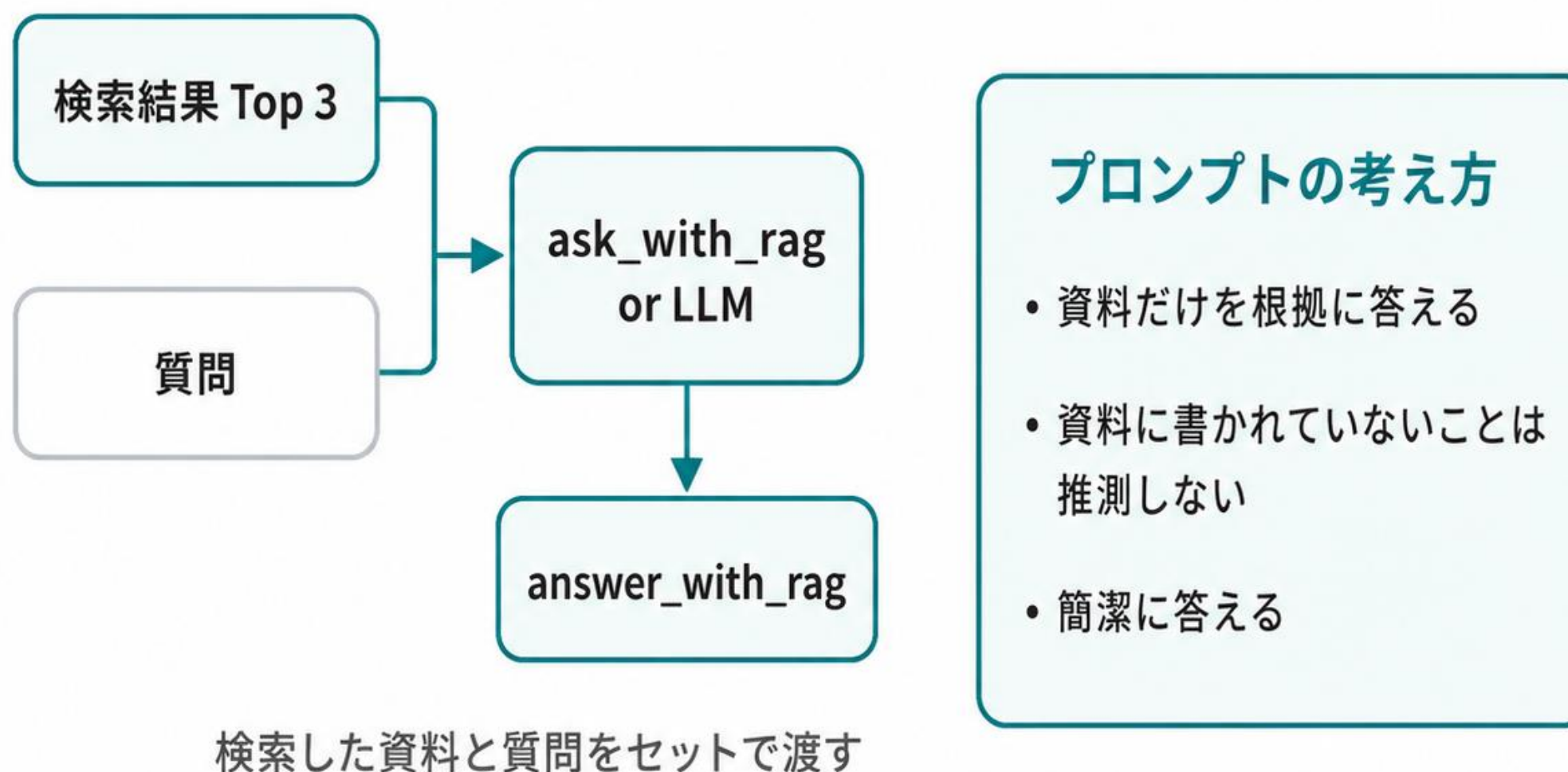
--- 検索結果 2 ---

... 日時：2020年9月2日（水）～ 4日（金） ...

答えの根拠が入っているかを見る

3-2-g Step 5 RAGありで同じ質問をする

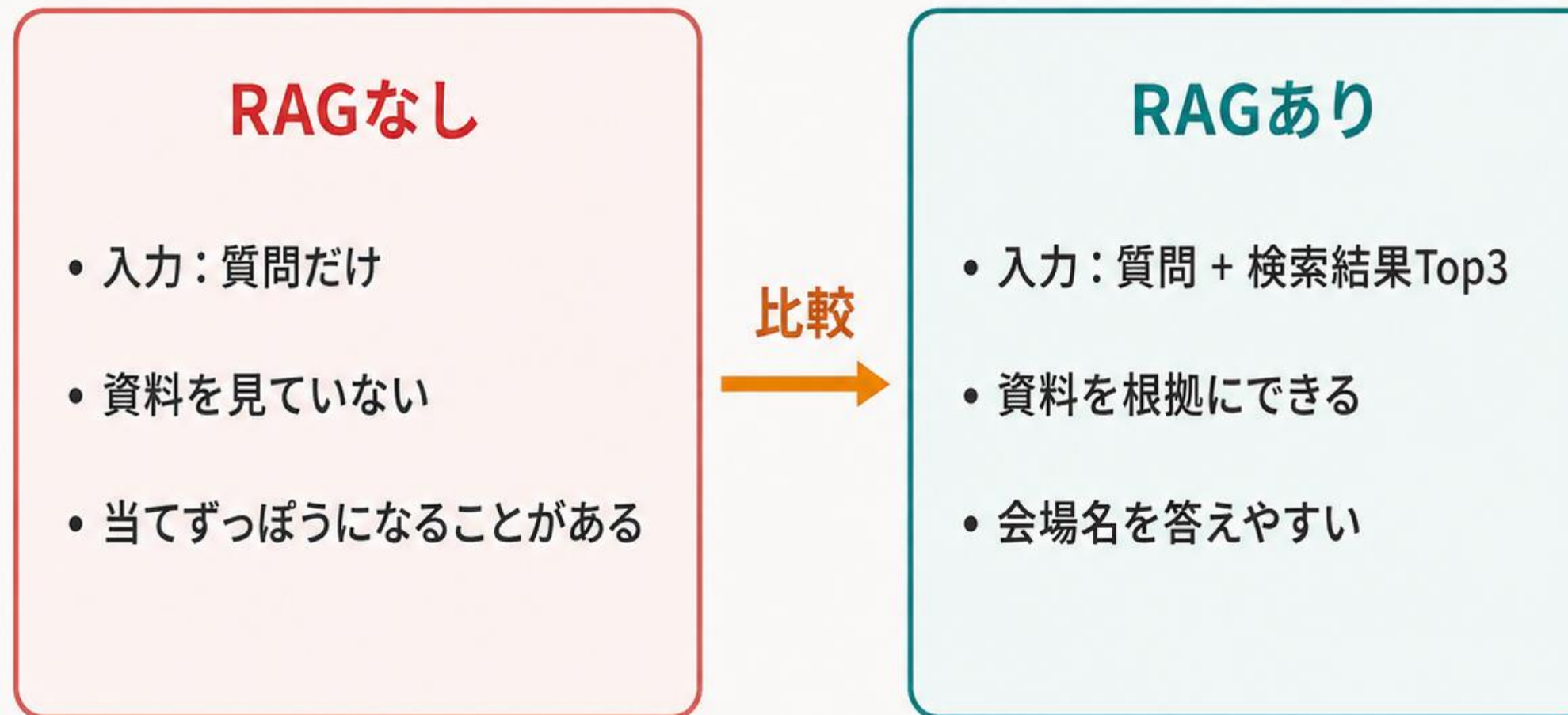
質問 + 検索で見つけた資料と一緒にLLMへ渡す



LLMが突然賢くなったのではなく、
答えるための資料を渡した

3-2-h Step 6 RAGなし / RAGありを比べる

何が変わったかを並べて確認する



答えのうまさより、根拠の有無に注目

見るポイント: RAGありの回答が、検索した資料を根拠にしているか

3-2-i まとめと自分で試す

資料だけでも流れを追えば成功

質問する → 関係する資料を探す → その資料と質問をLLMに渡す → 回答する



今日の用語

- **チャンク**：長い資料を分けた小さなかたまり
- **Embedding**：文章の意味を比較しやすい数値
- **RAG**：検索した資料をLLMに渡してから答えさせる方法



最後に試す質問例

- HI2020はいつ開催されましたか？
- HI2020の会場名を教えてください
- HI2020の原稿締切はいつですか？



コードを暗記する必要はありません。仕組みが分かればOK



ReAct

(20分)

思考と行動を組み合わせで、
より高度なタスクを実行できるAIの仕組みを学びます

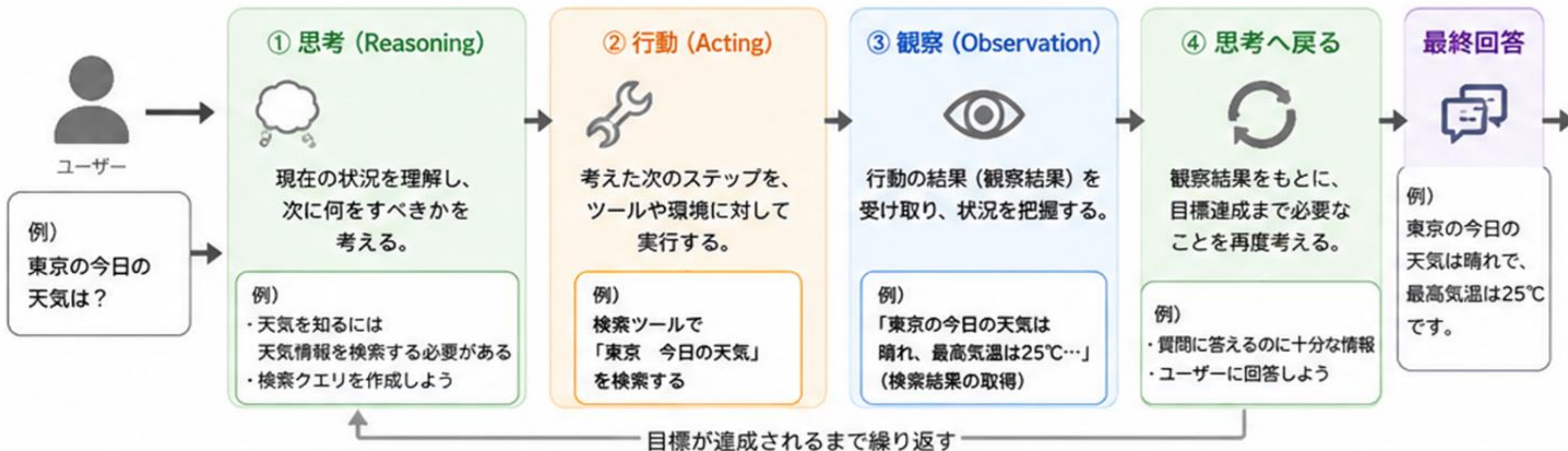
ReAct (Reasoning and Acting) とは？

LLMが「考える (Reasoning)」と「行動する (Acting)」を交互に行うことでより賢く、正確にタスクを解決するためのフレームワーク

例：天気を調べる

ReActの基本サイクル

思考 (Reasoning) → 行動 (Acting) → 観察 (Observation) を繰り返す



LLM学習ハンズオン

そもそも機械学習のお作法
訓練データ、検証データ、テストデータ
クロスバリデーションとか。機械学習一般の説明をいれるか。

一応ここでBERTとかGPTの学習（結局Transformerを別々に学習しているのだよさそう）

5

LoRA

(40分)

効率的にモデルをファインチューニングし、
自分専用のLLMを作ります

LoRAとは？

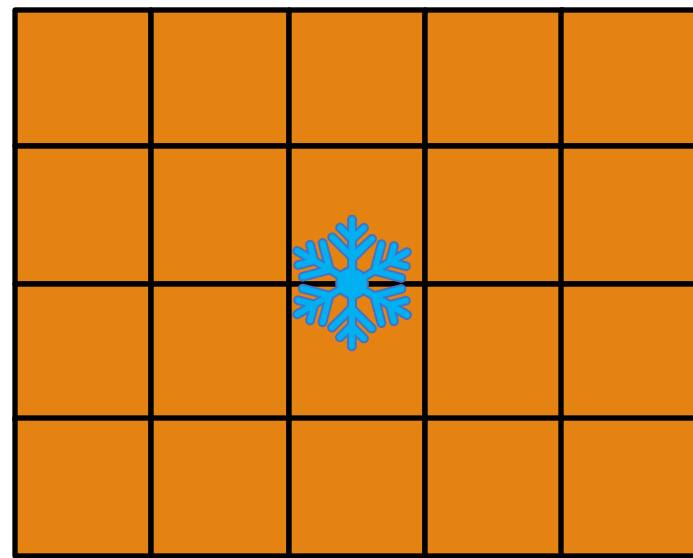
- ◆ Low-Rank Adaptationの略
- ◆ 大規模モデルのパラメータを固定し、
軽量アダプターで挙動を変える
- ◆ 少ない計算資源で効率的にファインチューニング可能

LoRA 

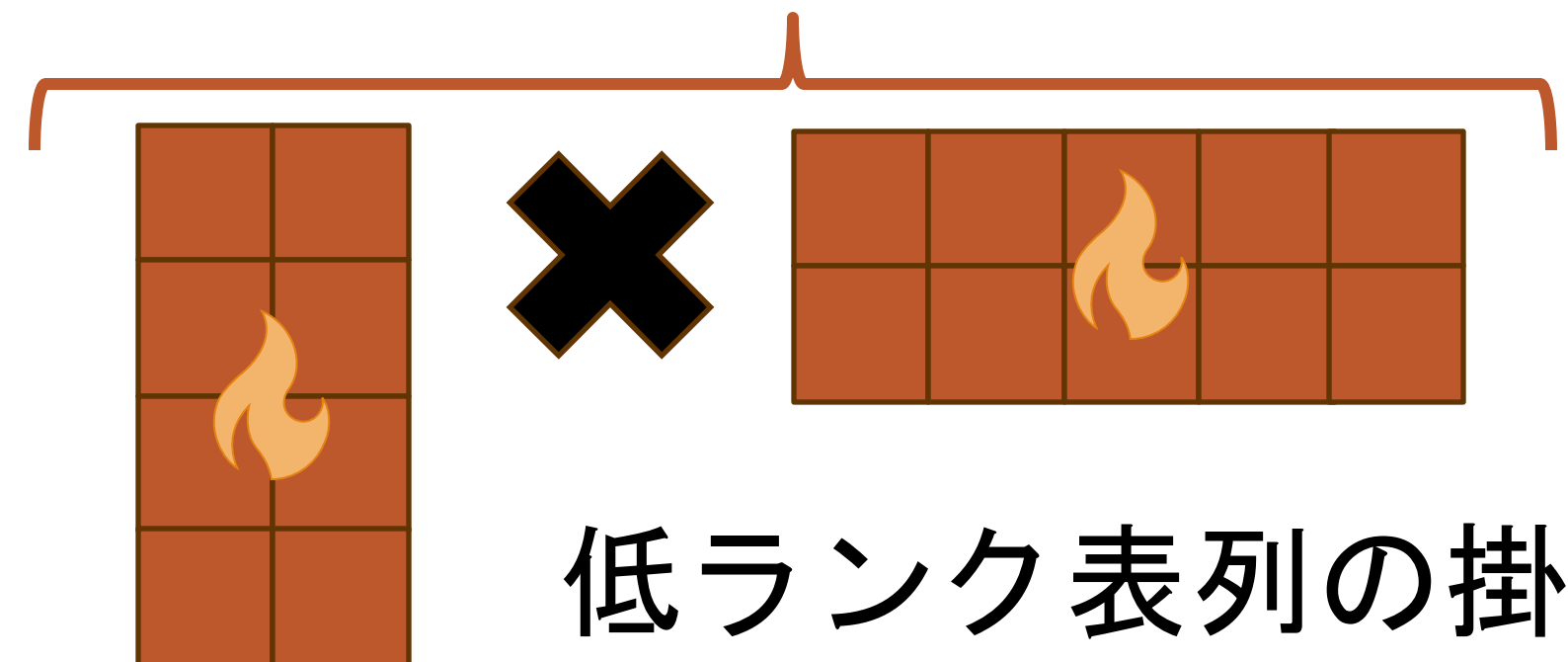
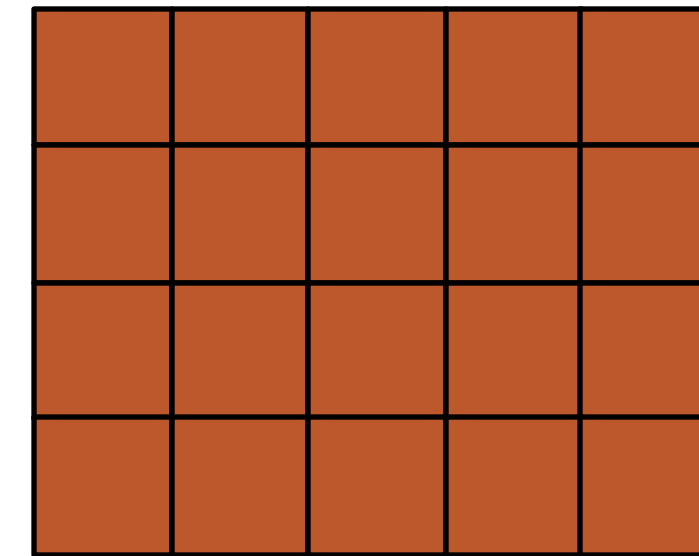
ベースの大規模モデル 

LoRAの仕組み

- ◆元の重み行列Wに、低ランク分解した更新量を加える
ベースのモデル LoRAアダプター



学習しない



低ランク表列の掛け算

LoRAのメリット

ファイルチューニングの**数%のコスト**で実現！

学習パラメータが大幅に削減される

GPUメモリ使用量を抑えられる

学習が高速でコストが低い

失敗のタスクに対してアダプタを使い分け可能

学習の流れ

1. データセットの準備・トークン化

2. ベースモデルの読み込み

3. LoRA設定の作成

4. LoRAアダプタを付与

5. 学習（アダプタのみ更新）

6. アダプタの保存・推論

1

LoRAチュートリアル

～ 30分で「ござる」口調のLLMをつくる ～



明日の予定を
短く説明してください。

この質問に、
「ござる」口調で答える
LLMを、LoRAを使って
作ります。

このチュートリアルでは、LoRAを使って
既存のLLMに小さな追加パラメータだけを学習し、
「ござる」口調で答えられるようにする方法を体験します。

1 LoRAチュートリアル

～ 30分で「ござる」口調のLLMをつくる ～



明日の予定を
短く説明してください。

このチュートリアルでは、LoRAを使って既存のLLMに小さな追加パラメータだけを学習し、「ござる」口調で答えられるようにする方法を体験します。

この質問に、
「ござる」口調で答える
LLMを、LoRAを使って
作ります。

1 / 8

2 LoRAとは？

LLM本体はほとんど固定したまま、小さな追加パラメータだけを学習する方法です。

事前学習済み
LLM
(ほぼ固定)

+

LoRA
(小さな追加パラメータ)



この部分だけを学習

- ・ 少ない学習データで効率的に学習できる
- ・ 既存のモデルを大きく変更しない
- ・ 文体・出力形式・タスクの解き方などを寄せるのに適している

2 / 8

3 今日の流れ

次の手順で、「ござる」口調のLLMを作ります。

- 1 LoRA学習前のLLMに質問する
- 2 「ござる」口調の小さな学習データを用意する
- 3 ベースモデルにLoRAを付ける
- 4 LoRAだけを短く学習する
- 5 LoRA学習後に同じ質問をして比較する

30分で
LoRAの基本的な流れを
実際に体験してみましょう。



3 / 8

4 学習前のLLMに質問する

まずは、元のLLMに質問してみましょう。



明日の予定を
短く説明してください。



LLM
(学習前)

明日は午前中に会議があります。
午後は資料作成を行い、
夕方にジムに行く予定です。

特別な口調ではなく、通常の回答が生成されます。

4 / 8

5 「ござる」口調の学習データを用意する

「質問 → ござる口調の回答」のペアをいくつか用意します。(今回は小さなデータセットでOKです)

質問	回答 (ござる口調)
自己紹介をしてください。	拙者はAIでござる。よろしくお揃いするでござる。
好きな食べ物は？	拙者はラーメンが好きでござる。
明日の予定を短く説明してください。	明日は午前中に会議、午後は資料作成、夕方にジムに行く予定でござる。
...	...

数十～数百件程度の
データでも、
文体の学習には
十分です。



5 / 8

6 ベースモデルにLoRAを付ける

事前学習済みのLLMに、LoRAモジュールを追加します。ベースモデル本体は変更しません。

事前学習済み
LLM
(固定)

+

LoRA
モジュール
(はじめはランダム)



LoRA付きLLM



LoRAは、特定の層に
低ランク行列を追加するだけなので、
パラメータ数が非常に少なくて
済みます。

6 / 8

7 LoRAだけを短く学習する

用意した「ござる」口調のデータを使って、LoRAのパラメータだけを学習します。

学習データ
(質問→ござる口調の回答)



学習
(数分～数十分)

LoRA
(この部分だけ更新)



- ・ ベースモデルのパラメータは更新しない、
- ・ LoRAのパラメータだけを学習する
- ・ 短時間・少ないデータでも文体を学習できる

7 / 8

8 学習後に同じ質問をして比較する

もう一度、同じ質問をして、回答を比較してみましょう。



明日の予定を
短く説明してください。



LLM
(学習前)

明日は午前中に会議があります。
午後は資料作成を行い、
夕方にジムに行く予定です。



LLM + LoRA
(学習後)

明日は午前中に会議、午後は資料作成、
夕方にジムに行く予定でござる。

このように、LoRAを使うことで、
少ない学習量で特定の文体に
寄せることができます。
LoRAは、文体・出力形式・タスクの
解き方などを調整したいときにも
よく使われます。

8 / 8

LoRA演習

ござるデータだっけ？

さようなら